

Ejercicio 5

Se necesita generar una permutación random de los n primeros número enteros. Por ejemplo $[4,3,1,0,2]$ es pero $[0,4,1,2,4]$ no lo es, porque un número está duplicado (el 4) y otro no está (el 3). Presentamos tres algoritmos para solucionar este problema. Asumimos la existencia de un generador de número random, $ran_int(i, j)$ el cual genera en tiempo constante, enteros entre i y j inclusive con igual probabilidad (esto significa que puede retornar el mismo valor más de una vez). También suponemos el mensaje $swap()$ que intercambia dos datos entre sí.

```
public class EjercicioPermutaciones {
    private static Random rand = new Random();

    public static int[] randomUno(int n) {
        int i, x = 0, k;
        int[] a = new int[n];
        for (i = 0; i < n; i++) {
            boolean seguirBuscando = true;
            while (seguirBuscando) {
                x = ran_int(0, n - 1);
                seguirBuscando = false;
                for (k = 0; k < i && !seguirBuscando; k++)
                    if (x == a[k])
                        seguirBuscando = true;
            }
            a[i] = x;
        }
        return a;
    }

    public static int[] randomDos(int n) {
        int i, x;
        int[] a = new int[n];
        boolean[] used = new boolean[n];
        for (i = 0; i < n; i++) used[i] = false;
        for (i = 0; i < n; i++) {
            x = ran_int(0, n - 1);
```

```

        while (used[x]) x = ran_int(0, n - 1);
        a[i] = x;
        used[x] = true;
    }
    return a;
}

public static int[] randomTres(int n) {
    int i;
    int[] a = new int[n];
    for (i = 0; i < n; i++) a[i] = i;
    for (i = 1; i < n; i++) swap(a, i, ran_int(0, i - 1));
    return a;
}

private static void swap(int[] a, int i, int j) {
    int aux;
    aux = a[i]; a[i] = a[j]; a[j] = aux;
}

/** Genera en tiempo constante, enteros entre i y j con igual probabilidad.
 */
private static int ran_int(int a, int b) {
    if (b < a || a < 0 || b < 0)
        throw new IllegalArgumentException("Parámetros inválidos");

    return a + (rand.nextInt(b - a + 1));
}

public static void main(String[] args) {
    System.out.println(Arrays.toString(randomUno(1000)));
    System.out.println(Arrays.toString(randomDos(1000)));
    System.out.println(Arrays.toString(randomTres(1000)));
}
}

```

- a. Analizar si todos los algoritmos terminan o alguno puede quedar en loop infinito.
- b. Describa con palabras la cantidad de operaciones que realizan.

a. randomUno

El algoritmo puede entrar en loop infinito porque *ran_int* puede generar número repetidos. Por ejemplo, cuando $i = 1$, *ran_int* puede retornar 0 o 1. Si retorna 0, sería un valor repetido de la iteración anterior ($i = 0$) y entraría en loop infinito.

randomDos

El algoritmo puede entrar en loop infinito porque existe la posibilidad de que *ran_int* genere números repetidos. Por ejemplo, en la segunda iteración ($i = 1$), puede generar el valor 0 todas las veces y nunca saldría del while.

randomTres

El algoritmo no entra en loop porque, con el método swap, se intercambian posiciones.

b. randomUno

Como en el peor de los casos se ejecuta infinitas veces, la cantidad de operaciones será infinita.

randomDos

Como en el peor de los casos se ejecuta infinitas veces, la cantidad de operaciones será infinita.

randomTres

Realiza una cantidad de operaciones fija y predecible, sin importar si es el mejor o el peor de los casos El primer for hace exactamente n asignaciones para llenar el arreglo con $[0, 1, 2, \dots, n - 1]$. El segundo for se ejecuta exactamente $n - 1$ veces. En cada iteración hace una llamada a *ran_int* y un swap (el cual consume 3 asignaciones internas).